* Exception / Exception Handling :-

1) Normal termination:

```
package Exception
public class Demo
{
    // normal termination
public static void main (String[] args) {
    System.out.println ("main started");

    Sopln ("main ended");
}
// when entire entire. executes successfully
    // till the last instruction
    // they we say it is normal termination
}
```

o/p :-

main started
main Ended.

2) Force termination

```
package Exceptions
public class Demo {
    // force termination
public static void main (String[] args) {
    Sopln ("main started");
// Jvm termination the program immediately when
this statement executes
```

25/11/21

// All remaining code after this line will not run
System.exit(0);
Sopln("main Ended");
}

// When a programmer explicitly writes code to terminate the program
// it is called force termination
}

O/P:
main Started

3) Abrupt termination

```
package ExceptionIntro;
public class Demo {
    // Abrupt ~~Exc~~ termination
    public static void main(String[] args) {
        Sopln("main Started");
        Sopln(10/0);
        Sopln("main Ended"); // ArithmeticException
    }
}
```

O/P:-
main started.
Exception in thread "main" java.lang~~.reflect~~.ArithmeticException:

18/11/2025

* **Exception!-**

It is an unexcepted event or a problem that occurs during execution ~~of~~ of a program.

When an exception occurs program execution will be abruptly terminated & from the line where exception occured [if not handled].

Eg:-
ArithmeticException
NullPointerException

Code:-

```
public class Demo
{
    public static void main (String [] args)
    {
        Sopln("main Started");
        String s = null;
        Sopln (s.charAt(0)); //Null PointerException
        Sopln ("main Ended");
        //When we try to access any member using null or reference we get NullPointerException.
    }
}
```

* **Exception handling :-**

In java exceptions are handled by using try-catch block.

Syntax :-

```
try{
    //instruction which may given exception
}
catch(Exception_name variable) // handler code
{


}
```

We use try block to write the instruction which may cause ~~exe~~ exception

Catch block is used to handle the exception if occured.

Eg :-

```
class Demo
{
    public static void main(String[] args)
    {
        Sopln("main started");
        String s=null;
        try{
            Sopln("try started");
            Sopln(s.charAt(0));
            Sopln("try ended");
        }
```

```
catch (Exception e)
{
    System.out.println("catch exception");
    System.out.println("exception handled");
    System.out.println("catch ended");
}
    System.out.println("main ended");
}
}
```

If any exception occurs in any line inside try block, control will directly go to the catch block without executing further instructions inside try block.

If the catch block is able to handle the exception occured catch block will get executed without abrupt termination.

If no exception occurs in try block, try block will be completely executed and catch block will never get executed.

Note:-

We can use n number of catch blocks along with one try block.

```
try{
    // instruction which may give exception
}
catch (Exception_name variable) // handler code
{

}
```

```
catch (Exception_name variable)
{

}
.....
number of catch block.
```

if 2 or more catch blocks are defined and exception occured control will go to the first catch block which can handle the occured exception.

* **try-catch-finally :-**

Syntax:-

```
try{
    // Instruction which may give exception
}
catch (Exception_name variable) //Handler Code
{

}
Finally {
    //always executed
}
```

Eg:-

```
public class Demo
{
  public static void main (String [] args)
  {
    Sopln ("main started");
    try {
      Sopln ("try started");
      Sopln (10/0);
      Sopln ("try ended");
    }
    catch (NullPointerException e) (OR) (ArithmeticException e)
    {
      Sopln ("exception handled");
    }
    finally
    { fo
      Sopln ("Important instruction");
    }
    Sopln ("main ended");
  }
}
```

O/P:-

main started
try started
important instruction
~~Exception thread~~
Exception thread main java.lang.
ArithmeticException :-

output when Exception ~~befor~~ occurs and not handled

main started
try started
exception handled
~~in~~ imported instruction
main ended

output when exception occurs and handled

Output if

main Stare
try Start
5
try ended
Important
main ended

1) If e
will be ex
handle o
executed.
If +
exception
block will

2) If e
will neve

Syntax 2:-

+
}
Co
}
We c
catch bl

Output if exception doesn't occurs

main Started
try Started
5
try ended
Important instruction
main ended

1) If exception occurs in try block first catch block will be executed if if that catch block is able to handle occurred exception and then finally will be executed.

If the catch block is not able to handle the exception occured even through it is not handled finally block will get executed.

2) If exception doesn't occurs in try block catch block will never be executed only finally will be executed.

Syntax 2:-

```
try{
    // Instruction which may give exception
}
catch finally{
    // always executes.
}
```

We can use finally along with try block without using catch block. But exception will not be handled.

Eg:-

```
public class Demo
{
    public static void main(String [] args)
    {
        Sopln("main Started");
        try {
            Sopln(10/0);
        }
        finally {
            Sopln("Important instruction");
        }
        Sopln("main ended");
    }
}
```

O/P:-

main Started

Important instruction

Exception in thread "main" java.lang.ArithmeticException

Eg:-

```
public class Demo
{
    public static void main (String [] args)
    {
        Scanner sc = new Scanner (System.in);
        try{
            Sop ln (10/0);
        }
        catch (NullPointerException e).
        {
            Sop ln ("handler code");
        }
        finally
        { Sop ln ("close costly resources before terminating);
            sc.close();
        }
    }
}
```

O/P:-

close costly resources befor terminating.

ArithmeticException.

Note:-

finally block will get executed whether exception occured or not occured whether it is handled or not handled.

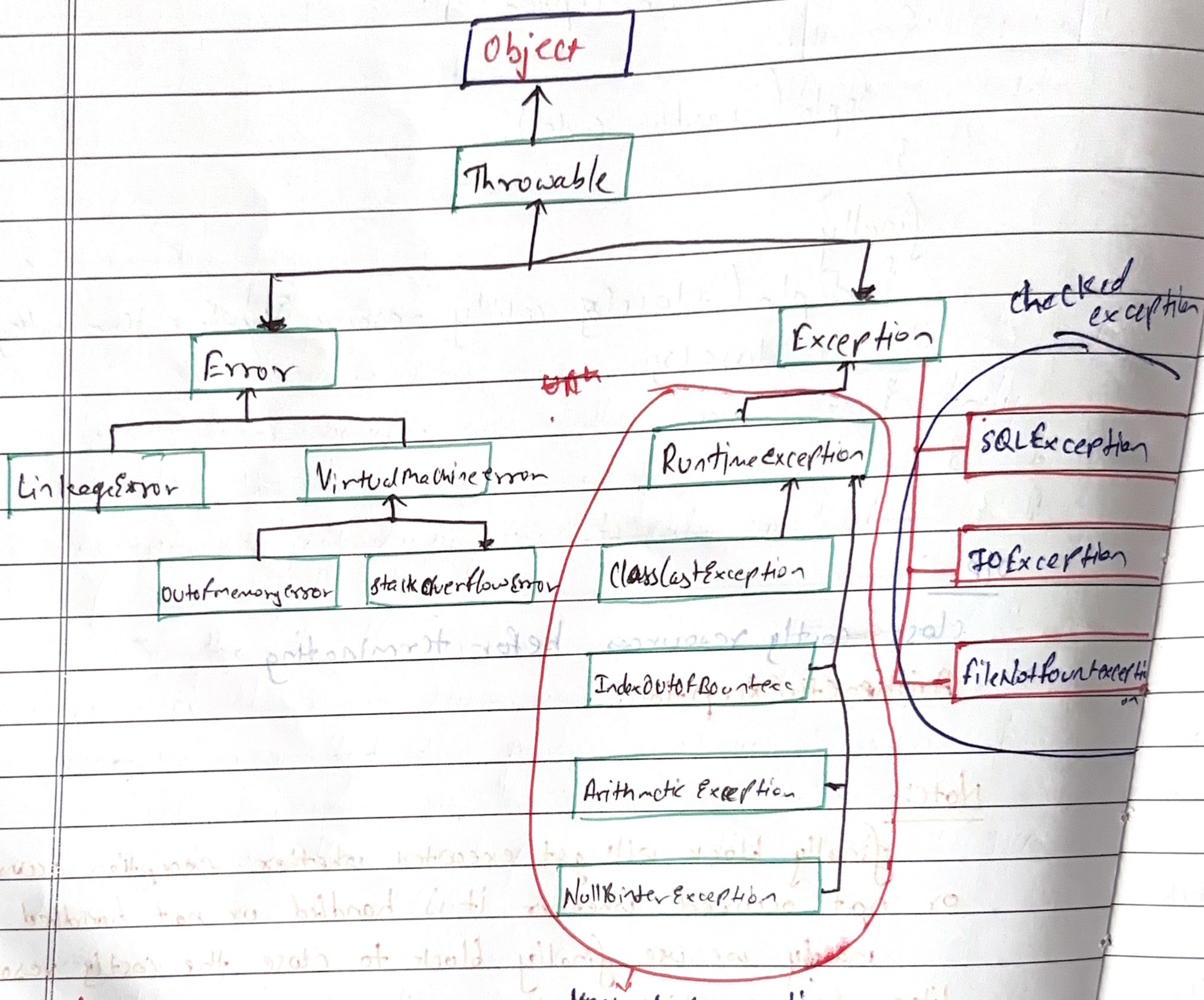
mainly we use finally block to close the costly resources like Scanner object and connection object.

19/11/2025

* What is exception with respect to java?

In java every exception is an object
For every exception object there is a built in class.

We have a hierarchy in $java.lang package for all the exceptions that occurs in java.



Note:-

Other than RuntimeException and its child classes all the other classes in the Throwable hierarchy are considered as checked exception.

* Throwable :-

Throwable is a super class of ~~every~~ error and exception
The throwable class defines the common properties and behaviour of error and exceptions.
The Throwable class is present in java.lang. package

* Important method in Throwable class :-

1] puble String getMessage();

2] public void printStackTrace();

* What exactly happens when an exception occurs in java?

When an exception occurs, one object of appropriate exception type will be created.
That object contains the details of exception, like ~~the~~ line number at which the exception occured, the method name and the class name ~~or~~ ant it also contains the ~~reason~~ reason for the exception.
This process is called as raising an exception.

Once the exception is raised or occured, JVM will check the exception is handled or not.
If exception is handled program execution will be continued
If the exception is not handled program execution will be abruptly stopped from the line where ~~exc~~ exception occured.

Eg:-

```
public class Demo:
{
  public static void main(String[] args)
  {
    Sopln("main Started");
    Sopln(10/0)
    Sopln("main Ended");
  }
}
```

create an object →

0x1

Object
Throwable
exception
RunTimeException
ArithmeticException

Contains → details of exception

ArithmeticException

* Exception object propogation:-

When an exception occurs in a method body the JVM look for the handler code in the same method body.

If the handler code is no fount in the method body the exception object propogation to caller method searching for the handler.

If the handler is found in the caller method the jvm executes the handler code and continues program execution from caller method this process continues until the handler is found somewhere in the entire stack.

If the handler is not present in the stack the JVM unwinds the stack and stops execution this is known as stack unwinding.

After unwinding the stack default exception handler calls the printStackTrace() s.

this method printStackTrace() prints all data related to exception.

* Definition:-

The mor
method to its
handled is kno

Eg:-

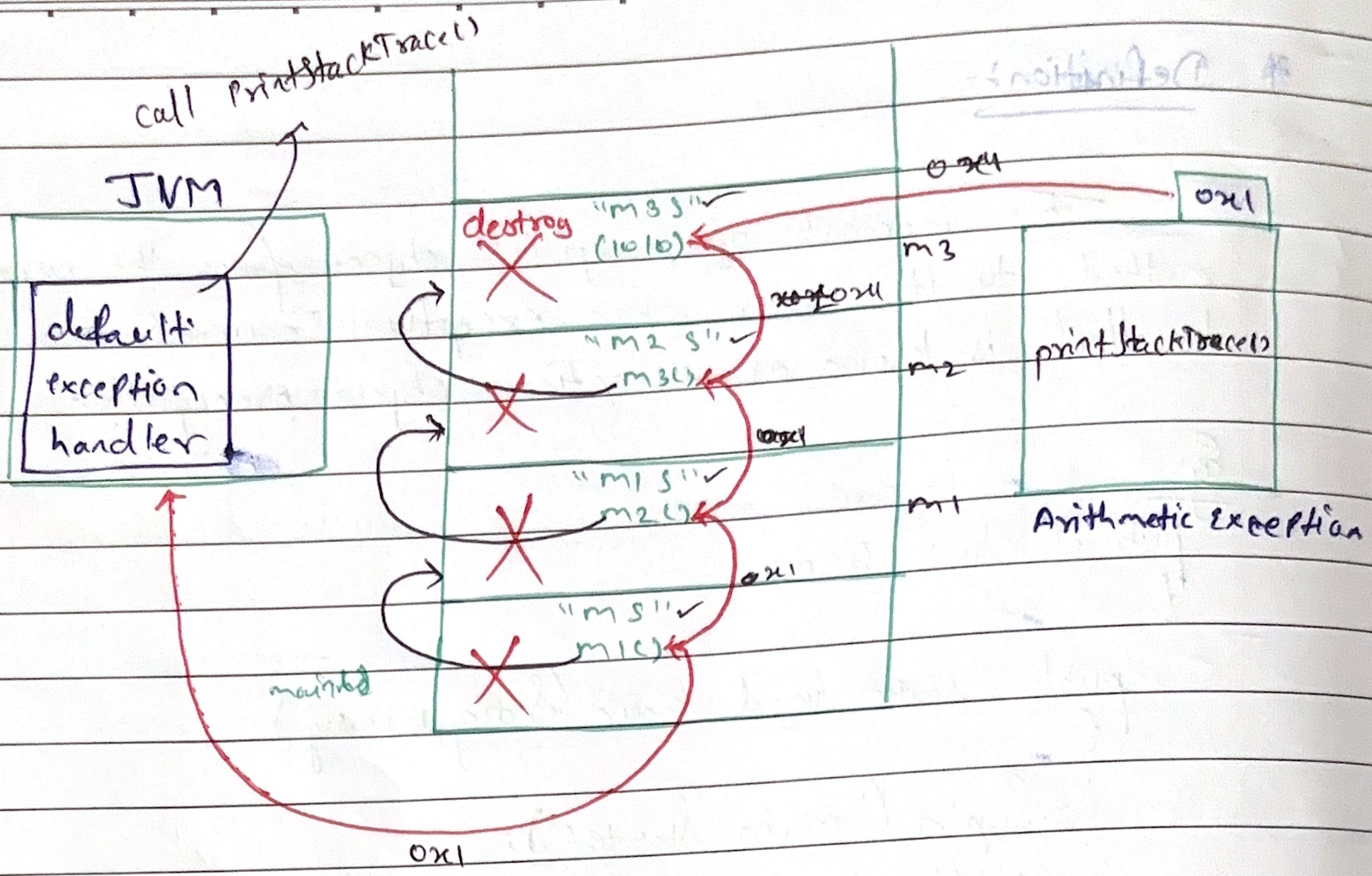
```
public class
{
public stat
{
Sopln
m1();
Sopln
}
public s
{
Sopln
m2();
Sopln
}
public st
{
Sopln
m3();
Sopln
}
public
Sopln
Sopln
}
```

* Definition :-

The moment of exception object from the current method to its caller when exception occured and not handled is known as exception object propogation.

Eg :-

```
public class Demo
{
  public static void main (String [] args)
  {
    Sopln ("main Started");
    m1();
    Sopln ("main Ended");
  }
  public static void m1(){
    Sopln ("m1 Started");
    m2();
    Sopln ("m1 Ended");
  }
  public static void m2(){
    Sopln ("m2 Started");
    m3();
    Sopln ("m2 Ended");
  }
  public static void m3(){
    Sopln ("m3 Started");
    Sopln (10/0);
    Sopln ("m3 ended");
  }
}
```



* Types of Exception :-

1] Unchecked Exception.
2] Checked Exception.

1] Unchecked Exception :-

Unchecked Exceptions are the exceptions which are not identified by the compiler.

All the subclasses to the RuntimeException class are unchecked exceptions.

ii) checked Exception :- Compile aware ~~exctpi~~ Exception.

The exceptions which are identified by the compiler during ~~compiat~~ compilation are checked exceptions.

If compiler identifies any checked exception we get

compile time error.

Until we write the handler code the CTE will not be gone.

All exceptions classes excepts RuntimeException hierarchy comes under checked exception.

They are identified during compile Time but occurs during runtime only.

```
package ExceptionIntro.Types;
import java.io.File;
import java.io.IOException;
public class Demo
{

    public static void main (String [] args)
    {
        System.out.println("main started");
        File f = new File("c:\\basic");
        //this instruction may throw IOException
        //since it ~~shows~~ throws IOException which is check exception
        //we get CTE.
        //we must and should handle this using try-catch block
        f.createNewFile(); //CTE
        System.out.println("main ended");
    }
}
```

After writing handler code

```
package ExceptionIntro.Types;
import java.io.File;
import java.io.IOException;
public class Demo
{

    public static void main(String[]args)
    {
        ~~File f=n~~
        File f = new File("c:\basic");

        try{
            f.createNewFile();
        }
        cater (IOException e)
        {

        }
    }
}
```

Note:-

checked exceptions are not propogated automatically to the caller method

Only unchecked exceptions are propogated automatically.

* Throws:-

Throws
method declar
It is
from called
It spec
in the curren
handled in th

Example prog

```
package Exceptio
import java.i
import java.i
import java.
class Demo
{
    public stat
    {
        Sopln (
        File f
        f.creat
        Sopln (
    }
    public s
    {
        Sopln(
        m3();
        Sopln (
    }
```

* Throws:-

Throws is a declaration keyword used in the method declaration or constructor declaration.

It is ~~us~~ used to propogate the exception object from called method to caller method.

It specified that the cherked exception may occurs in the current method body is not handled it must be handled in the caller method.

Example program with checked exception.

```
package ExceptionIntro.Types;
import java.io.Files;
import java.io.FileNotFoundException;
import java.io.IOException
class Demo
{
  public static void m3() throws IOException
  {
    sopln ("m3 started");
    File f = new File ("c:\basic");
    f.createNewFile();
    sopln ("m3 Ended");
  }
  public static void m2() throws IOException
  {
    sopln ("m2 started");
    m3();
    sopln ("m2 Ended");
  }
```

```
public static void m1() throws IOException
{
    sopln("m1 started");
    m2();
    sopln("m1 Ended");
}
public static void main(String[] args)
{
    sopln("main Started");
    m1();
    sopln("main Ended");
}
}
```

Date: 21/11/2025

* Custom Exception or used defined exceptions:-

Exceptions which are created by the programmer is known as user defined exception.

In java we can create user defined exception objects by using throw keyword.

* Throw:-

Throw is a keyword which is used to ~~the~~ create and raise custom exception object.

Throw keyword can be used only inside blocks.

By using throw keyword we can throw ~~etc~~ checked exceptions, unchecked exceptions and custom exception also.

Syntax:-

throw

Eg:-

throw ne

* Steps to crea

Create
throwable
Create
to throw the
to displ
we can creat
message inside
can call pass

Note:-

Using
type objects

Eg:-

~~eto~~ packag
public class
{
// to p
creating a
// this
throwable
// thro

Syntax:-

throw ~~Trow~~ ThrowableTypeObject;

Eg:-

throw new ArithmeticException();

* Steps to create custom exception:-

Create a class and inherit any class which is of throwable type.

Create the object of the class and use throw keyword to throw the object.

to display the reason for exception if not handled, we can create a parameterized constructor which accepts String message inside the custom exception class and using super() we can call pass this message to Throwable class.

Note:-

Using throw keyword we can throw only throwable type objects.

Eg:-

```
package custom;
public class NotEligibleToMarryException extends RuntimeException.
{
    // to print the message along with the exception we are creating a constructor
    // this message will be passed to detailsMessage variable inside throwable class.
    // through constructor chaining process using super();
```

```
public NotEligibleToMarryException(String message){
    super(message);
}
}

class Driver
{
public static void main(String[] args)
{
    Scanner sc = new Scanner(System.in);
    System.out.println("Enter the age");
    int age = sc.nextInt();

    if(age<18)
    {
        throw new NotEligibleToMarryException("you are still a
                                    kid and can't make kids");
    }
    else
        System.out.println("You can marry at your own risk");

    System.out.println("main Ends");
}
}
```

O/P:-

```
Enter the age
16
Exception in thread "main" Custom.NotEligibleToMarryException: You are still
a kid and can't make kids.
```

```
public class
{
public stat
{
    System.out.println(
    throw n
}
}

public cla
{
public
{
    //if we
    //except
    //compil
}
```

Note :-

throw

that block

public

//t

th

//sig

keyword

//wo

during

```
public class Driver4
{
    public static void main (String [] args)
    {
        Sopln ("main Started");

        throw new NotEligibleToMarryException ("not eligible");

        Sopln ("main Ended"); → CTE [unreachable code].

        //if we are writing any code after throwing an
        //exception object using throw keyword we get
        //compileTimeError stearting that unreachable code
```

Note :-

throw keyword should be last executable instruction within that block it if it is not handled

```
    }
}

public class Driver5
{
    public static void main (String [] args)
    {
        //throwing checked exception
        throw new IOException ();

        //if we are throwing checked exception using throw
keyword we get CTE.
        //we must and should handle the checked exception
during compile time
```

//until we handle the exception error will not be gone.
}
}

Note:-
Using throw keyword we can throw checked exception, unchecked exception as well as custom exception.

* Important Points:-

The Exception and throwable classes are partially checked and partially unchecked, even though compiler will consider them as checked exceptions only.

$

If we are creating custom exception class by inheriting checked exception or partially checked exception, the custom exception also will be considered as checked exception by the compiler.

If we are trying to throw the custom exception which is considered as checked exception we have to either handle it or propogate it by using throws keyword it works same like checked exception.

Eg:-
```
public class Driver6
{
  public static void main (String[] args)
  {
    try{
      Sopln (10/0);
    }
```

```
catch (exceptio
{

}
catch (Arithme
{

}
catch (NullPoin
{

}
} // When w
with try blo
be maintained
on the top
if we keep
handle any
We get C
unreachable
}
```

```
catch (Exception e)
{

}
catch (ArithmeticException ae) //CTE
{

}
catch (NullPointerException ne) //CTE
{

}
```

// When we are using a number of catch blocks along with try block the order of the catch block ~~shoo~~ should be maintained such that always child should be always on the top and parent should be at the bottom because if we keep parent catch block on top that itself will handle any type of exception occured.

We get CTE for botton catch blocks stating that unreachable catch block.